

Upload API Notes — AI-Powered Exam Preparation System

1. Introduction

This document explains the implementation of the Upload API for the AI-Powered Exam Preparation System.

The Upload API is the first component of the complete RAG (Retrieval-Augmented Generation) pipeline.

The main objective of this module is:

- Receive PDF files from users
- Validate uploaded files
- Store documents locally
- Generate unique document IDs
- Prepare documents for parsing and embedding

This module acts as the entry point of the document ingestion pipeline.

2. Current System Architecture

The implemented structure currently focuses only on document upload functionality.

```text id="5gv5pc" User ↓ FastAPI Upload API ↓ Validate PDF ↓ Generate Document ID ↓ Store PDF Locally ↓ Return Metadata

```

3. Current Project Structure

```text id="ojjlmw"
Exam_Intelligence/
|
|— app/
|   |
|   |— api/
|       |— __init__.py
|       |— upload.py
|       |
|       |
```

```
| | | models/
| | | | | __init__.py
| | | | | document.py
| | |
| | | services/
| | | | | __init__.py
| | | | | parser.py
| | | | | chunker.py
| | | | | embedder.py
| | | | | vectordb.py
| | |
| | | utils/
| | | | | __init__.py
| | | | | helpers.py
| | |
| | | main.py
|
| uploads/
|
| chroma_db/
|
| docs/
|
| requirements.txt
|
| README.md
```

4. Understanding FastAPI

What is FastAPI?

FastAPI is a modern Python web framework used for building APIs.

It is specifically designed for:

- Fast backend development
- High performance APIs
- Automatic documentation
- Async support
- Type validation

FastAPI is widely used for:

- AI systems
- RAG applications
- Machine Learning APIs
- Backend services

5. Why FastAPI Is Used in This Project

FastAPI is ideal for RAG systems because:

Feature	Benefit
High Speed	Handles many requests efficiently
Async Support	Useful for AI pipelines
Automatic Swagger UI	Easy API testing
Type Validation	Reduces bugs
Clean Syntax	Easy to understand

6. Required Libraries

The following libraries were installed:

```
```bash id="f8r2xm" pip install fastapi uvicorn python-multipart
```

```

7. Purpose of Each Library

FastAPI

Used to:

- create APIs
- define routes
- handle requests

Example:
```

```
```python id="wlycqz"
from fastapi import FastAPI
```

Uvicorn

Used as an ASGI server to run FastAPI applications.

Example:

```
```bash id="yexq7x" uvicorn app.main:app --reload
```

```

python-multipart

Required for handling file uploads.

Browsers upload files using:

```text id="f3t2ja"
multipart/form-data
```

Without this package:

- file uploads will fail

8. Understanding FastAPI Application

File:

```
```text id="x6e1b8" app/main.py
```

Code:

```
```python id="h8vdt5"
from fastapi import FastAPI
from app.api.upload import router as upload_router

app = FastAPI()
```

```
app.include_router(upload_router)

@app.get("/")
def home():
    return {"message": "Server Running"}
```

9. Explanation of main.py

Creating FastAPI App

```
```python id="8gmjdz" app = FastAPI()
```

```
This creates the main backend application.
```

```

```

```
Connecting Router
```

```
```python id="80z2qa"
app.include_router(upload_router)
```

This connects APIs from:

```
```text id="sxx75g" upload.py
```

```
to the main application.
```

```

```

```
Root Endpoint
```

```
```python id="we8b98"
@app.get("/")
```

Creates a GET API route.

When users visit:

```
```text id="a5j7m3" http://127.0.0.1:8000/
```

they receive:

```
```json id="ex0p3f"
{
  "message": "Server Running"
}
```

10. Understanding API Routes

Example:

```
```python id="g1o8q5" @app.get("/")
```

Meaning:

Part	Meaning
@app	FastAPI app
get	HTTP GET request
"/"	API route

---

# 11. What Is Swagger UI?

Swagger UI is an automatically generated API testing interface.

FastAPI automatically creates Swagger UI at:

```
```text id="yjvrpn"
http://127.0.0.1:8000/docs
```

12. Purpose of Swagger UI

Swagger UI allows developers to:

- test APIs
- upload files
- send requests
- inspect responses

without writing frontend code.

This is extremely useful during backend development.

13. Why Swagger UI Is Important

Before building frontend applications, backend APIs can be tested independently.

Workflow:

``text id="vxvg4q" Swagger UI ↓ Test Upload API ↓ Verify Backend Logic

```
---

# 14. Upload API Implementation

File:

```text id="t6mqn9"
app/api/upload.py
```

---

## 15. Upload API Code

```
``python id="3jllwm" from fastapi import APIRouter from fastapi import UploadFile from fastapi import File
from fastapi import HTTPException
```

```
import os import uuid import shutil
```

```
router = APIRouter()
```

```
UPLOAD_DIR = "uploads"
```

```
os.makedirs(UPLOAD_DIR, exist_ok=True)
```

```
@router.post("/upload") async def upload_pdf(file: UploadFile = File(...)):
```

```
 if file.content_type != "application/pdf":
 raise HTTPException(
 status_code=400,
 detail="Only PDF files are allowed"
```

```

)

 document_id = str(uuid.uuid4())

 document_folder = os.path.join(
 UPLOAD_DIR,
 document_id
)

 os.makedirs(document_folder)

 file_path = os.path.join(
 document_folder,
 file.filename
)

 with open(file_path, "wb") as buffer:
 shutil.copyfileobj(
 file.file,
 buffer
)

 return {
 "document_id": document_id,
 "filename": file.filename,
 "status": "uploaded"
 }

```

...

## 16. Understanding APIRouter

```
python id="7c15k8"
router = APIRouter()
```

APIRouter is used for modular API design.

Instead of placing all APIs in one file:

```
```text id="vcy1wa" main.py
```

multiple route files can be created:

```
```text id="0jtxpx"
```

```
upload.py
search.py
chat.py
parser.py
```

This improves:

- maintainability
- scalability
- code organization

---

## 17. Understanding UploadFile

```
```python id="t9r0k4" file: UploadFile = File(...)
```

UploadFile is used to receive uploaded files.

Benefits:

- memory efficient
- handles large files
- streams file data
- stores metadata

```
# 18. Understanding File(...)
```

```
```python id="mmsrjx"
File(...)
```

This indicates:

- the file is required

Without it:

- requests may be empty
-

## 19. File Validation

```
```python id="c7f3ow" if file.content_type != "application/pdf":
```

This validates uploaded file types.

Rejected files:

- PNG
- EXE
- ZIP
- JavaScript files

This improves:

- security
- consistency
- pipeline reliability

20. Understanding UUID

```
```python id="nkjw8j"
uuid.uuid4()
```

Generates unique document identifiers.

Example:

```
```text id="e2hxt6" 9f2b2db8-cdb7-4db4
```

Why needed?

Because multiple users may upload files with identical names.

21. Why Separate Folders Are Created

Structure:

```
```text id="bfxy0"
uploads/
```

```
└─ document_id/
 └─ file.pdf
```

Advantages:

- prevents overwriting
- simplifies document tracking
- improves scalability

---

## 22. Understanding Binary Write Mode

```
```python id="l4u7js" with open(file_path, "wb")
```

```
`wb` means:
```

```
- write binary
```

```
PDF files are binary files, not plain text.
```

```
---
```

```
# 23. Understanding shutil.copyfileobj()
```

```
```python id="50s5p8"
shutil.copyfileobj(
 file.file,
 buffer
)
```

This copies uploaded file data into local storage.

Workflow:

```
```text id="tlw90o" Uploaded PDF ↓ File Stream ↓ Local File
```

```
---
```

```
# 24. API Response
```

```
Example response:
```

```
```json id="g24h8h"
```

```
{
 "document_id": "6bafc7d2",
 "filename": "DBMS.pdf",
 "status": "uploaded"
}
```

This metadata will later be used for:

- parsing
- chunking
- embeddings
- retrieval

---

## 25. Running the Server

Command:

```
``bash id="n1x9vt" uvicorn app.main:app --reload
```

```

26. Understanding Uvicorn Command

app.main

Represents:

```text id="lc6l1m"
app/main.py
```

:app

Represents:

```
``python id="qps0zm" app = FastAPI()
```

```
---

## --reload
```

Automatically reloads server when files change.

Useful during development.

27. Current Workflow

The implemented workflow is:

```
```text id="6b0eq3"
```

User Uploads PDF

↓

FastAPI Receives File

↓

Validate PDF

↓

Generate UUID

↓

Create Folder

↓

Save PDF

↓

Return Metadata

---

## 28. Current Achievement

The following features have been successfully implemented:

```text id="icw8xg" ✓ FastAPI server ✓ Modular backend structure ✓ Swagger UI ✓ Upload API ✓ PDF validation ✓ UUID document tracking ✓ Local document storage

29. Why This Upload System Is Important

This Upload API is the first step of the complete RAG ingestion pipeline.

Future workflow:

```
```text id="dij3w0"
```

Upload PDF

↓

```
Parse PDF
 ↓
Extract Text
 ↓
Chunking
 ↓
Embeddings
 ↓
Vector Database
 ↓
Semantic Retrieval
```

Without upload functionality:

- parsing cannot begin
- embeddings cannot be generated
- vector storage becomes impossible

---

## 30. Next Planned Component

The next implementation phase will focus on:

### PDF Parser

Responsibilities:

- read stored PDFs
- extract page-wise text
- prepare data for chunking

Technology:

- PyMuPDF

Example output:

```
json id="mj1wmj"
{
 "pages": [
 {
 "page": 1,
 "text": "Database Management System..."
 }
]
}
```

```
]
}
```

This will become the second stage of the RAG ingestion pipeline.